Practical Session: Locomotion Software for Legged Robots, real experiments

Objectives

This practical session introduces the main components of a model-based locomotion software stack for a quadruped robot in simulation. There are several blocks/modules in a locomotion framework, we will do exercises separately on them and then put all the modules together and experience and do tests with the whole framework. The goal is to progressively build, tune, and test the following modules of the framework: Swing controller, Stance controller, Foothold generator, Gait scheduler.

Install the software

First repo:

```
git stash
git fetch origin
git checkout origin/feat/lesson_saturday
```

Very important: We now use the branch `feat/lesson_saturday`.
Very important: Every time you open a new terminal, you need to activate the conda environment first, see the README!

1 Exercise 1: Swing-Foot Trajectory Generator and Control

The swing-foot trajectory must move the foot smoothly from lift-off to the desired touchdown point. It should provide sufficient clearance above the terrain and have low touchdown velocity.

For a cubic trajectory:

$$p(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3.$$

Choose the coefficients to satisfy initial and final position and velocity boundary conditions. The file to modify is `Quadruped-PyMPC/quadruped_pympc/helpers/cubic_swing_trajectory.py` Startin and final position of the spline are already given in the main of this file, that you can launch with `python3 Quadruped-PyMPC/quadruped_pympc/helpers/cubic_swing_trajectory.py`

Hint: search for EXERCISE 1.1-1.4 in the file and implement between the dotted lines.

1. Generate horizontal motion from lift-off (initial point) to touchdown (final point).
2. Add a vertical swing-height profile - the robot should not slide the feet on the terrain!
3. Compare position, velocity, and acceleration continuity.
4. Verify that touchdown velocity is close to zero. Real robot can suffer from discontinuities!

For tracking the trajectory, a control law is needed (1):

$$\boldsymbol{\tau}_{sw,i} = \boldsymbol{J}_i^T \left[\boldsymbol{K}_p \left(\boldsymbol{p}^{\text{ref}} - \boldsymbol{p} \right) + \boldsymbol{K}_d \left(\dot{\boldsymbol{p}}^{\text{ref}} - \dot{\boldsymbol{p}} \right) \right].$$

A better control law can be obtained by compensating the gravitational effects and coriolis (2).

$$\boldsymbol{\tau}_{sw,i} = \boldsymbol{J}_i^T \left[\boldsymbol{K}_p \left(\boldsymbol{p}^{\text{ref}} - \boldsymbol{p} \right) + \boldsymbol{K}_d \left(\dot{\boldsymbol{p}}^{\text{ref}} - \dot{\boldsymbol{p}} \right) \right] + h(\boldsymbol{q}, \dot{\boldsymbol{q}}).$$

The file to modify is `Quadruped-PyMPC/quadruped_pympc/helpers/swing_trajectory_controller.py`. Run its main with `python3 Quadruped-PyMPC/quadruped_pympc/helpers/swing_trajectory_controller.py` HINT: search for EXERCISE 1.5-1.7 in the file and implement between the dotted lines.

1. Implement controller (1), and check the tracking of the spline trajectory; Tune the gains K_p and K_d to improve tracking - range [200-500 Kp], [5-20 Kd].
2. Implement controller (2), and check the tracking of the spline trajectory; Tune the gains K_p and K_d to improve tracking - range [200-500 Kp], [5-20 Kd].
3. These are the controllers you will use on the real robot, try to obtain a smooth action!

KP and KD can be tuned in The file to modify is `Quadruped-PyMPC/quadruped_pympc/config.py`, lines 202-203.

2 Exercise 2: Stance Controller

During stance, the desired contact force computed by the MPC is converted into joint torques through the foot Jacobian:

$$\boldsymbol{\tau}_{st,i} = -\boldsymbol{J}_i^T \boldsymbol{f}_i^{\text{des}}.$$

The Jacobian must be consistent with the frame in which the contact force is expressed.

Modify `Quadruped-PyMPC/quadruped_pympc/interfaces/wb_interface.py`, lines 372-377, search for "EXERCISE 2.1"

1. Extract the foot Jacobian from the robot model.
2. Take the GRF from the MPC.
3. Compute the joint torques using the Jacobian and the desired contact force.

Exercise 3: Gait Scheduler

The gait scheduler generates a binary contact state for each foot:

$$s_i(t) \in \{0, 1\},$$

where $s_i = 1$ denotes stance and $s_i = 0$ denotes swing.

Each gait is described by its cycle duration, duty factor, and phase offsets.

The file to modify is `Quadruped-PyMPC/quadruped_pympc/helpers/periodic_gait_generator.py` You can launch its main with `python3 Quadruped-PyMPC/quadruped_pympc/helpers/periodic_gait_generat` You can select the gait in `Quadruped-PyMPC/quadruped_pympc/config.py` in 217. Remember to change duty factor and step frequency in the config file as well there.

1. Generate the contact schedule for the crawl.
2. How many different types of crawl can you generate?
3. Change duty factor and gait frequency during simulation to find a stable gait.

Gait	Swing pattern	Typical use
Crawl	One foot swings at a time	Slow and statically stable walking
Pace	Left and right lateral pairs alternate	Fast lateral-pair gait
Trot	Diagonal pairs alternate	Dynamic walking and running
Bound	Front and hind pairs alternate	Fast locomotion

Exercise 4: Real robot

With all you changes done in these exercises, try to run the simulation: `python3 Quadruped-PyMPC/simulation/`
If it works, congrats, we can try it on the real robot!